

CS102 – Algorithms and Programming II
Programming Assignment 6
Fall 2025-2026

ATTENTION:

- Compress all of the Java program source files (.java) files into a single zip file.
- The name of the zip file should follow the below convention, where you replace the variables “Sec1”, “Surname” and “Name” with your actual section, surname and name:
CS102_Sec1_Asst6_Surname_Name.zip
- Upload the above zip file to Moodle by the deadline (if not, significant points will be taken off). You will get a chance to update and improve your solution by consulting to the TAs and tutors during the lab. You have to make the preliminary submission before the lab day. After the TA checks your work in the lab, you will make your final submission. Even if your code does not change in between these two versions, you should make two submissions for each assignment.

GRADING WARNING:

- Please read the grading criteria provided on Moodle. The work must be done individually. Code sharing is strictly forbidden. We are using sophisticated tools to check the code similarities. The Honor Code specifies what you can and cannot do. Breaking the rules will result in disciplinary action.

BIG INTEGER SORTING APPLICATION

For this lab, you will implement a Java console application to store and sort big integers. Although Java already supports such numbers via the `BigInteger` class, you are not allowed to use this class. Instead, you should implement your own class for storing such numbers. You may study the implementation of the `BigInteger` class for inspiration. For your `BigInteger` implementation, do not use a `String` to store the digits.

Your `BigInteger` class should:

- Support storing **non-negative integers** up to 200 digits.
- Include a constructor that initializes the number from a string input. The constructor should only use the last 200 digits if a longer string is supplied. Ensure that all characters of the input string are numeric; otherwise, throw an exception.
- Implement an add method that takes another `BigInteger` and returns the sum of the two numbers.
- Implement a subtract method that takes another `BigInteger` and returns the difference of the two numbers. If the first number is less than the second number throw an exception.
- Implement a multiply method that takes another `BigInteger` as input and returns the product of the two numbers. If one of the numbers has more than 20 digits, the method should use the recursive Karatsuba algorithm. Otherwise, it should use regular iterative multiplication.
- Implement a compare method that compares the integer with another `BigInteger`. This method should iterate over the digits of both numbers to determine the result. For example, if the most significant digit of one number is bigger, return a positive or negative integer depending on which is larger. Otherwise, check the following digits. If all digits are the same, return 0. Ensure your `BigInteger` class implements the `Comparable<BigInteger>` interface.

For this lab, you need to generate a list of big integers. For this, you will use a variation of John Von Neumann's Middle-square pseudorandom number generator. The modified algorithm starts with an initial seed number (X_0) with n digits and produces a sequence of numbers by applying the following operations:

$$X_{k+1} = \text{Middle}((X_k)^2 + X_k)$$

where *Middle* returns the middle n digits. Before applying the *Middle* make sure the number has at least $2n$ by padding it with leading zeros if necessary. Additionally, if the extracted middle number has fewer than n digits (not counting leading zeros), shift the n digit window to the left until the number has n proper digits (in other words, a non-zero most significant digit).

Your application should first prompt the user to enter an initial seed number and the number of big integers to generate. The program will then generate a sequence of big integers using the modified Middle-square pseudorandom number generator described above. Each generated number should be treated as a big integer and stored in a list for further operations.

In addition to generating numbers, your application should allow the user to sort the generated list using several sorting algorithms. The lab focuses on implementing and measuring the performance of the following sorting algorithms:

- Bubble sort
- Merge sort
- Quicksort
- Quicksort with a random element as pivot
- Quicksort with median of first, middle, and last elements as pivot

For each sorting algorithm, implement a generic sort method that accepts a Comparable array so that your method can sort any array of objects that implement the Comparable interface. You can also test your sorting algorithms with other types, such as integers, strings, or any other objects that implement Comparable.

After each sorting operation, the program should display the first 50 numbers of the original list and the sorted list, along with the time taken to perform the sort. Your program should also verify that the numbers are correctly sorted and indicate to the user if the sort was successful. For this, you can check if the sorted sequence is non-decreasing.

Your program should provide a menu for selecting the sorting algorithm. After completing a sorting operation, the menu should be displayed again, allowing the user to choose another algorithm or exit the program.

For measuring the time taken by your operations, you can use the `System.currentTimeMillis()` method as follows:

```
long startTime = System.currentTimeMillis();
// perform the sorting operation here
long endTime = System.currentTimeMillis();
double elapsedSeconds = (endTime - startTime) / 1000.0;
```

Preliminary Submission: You will submit an early version of your solution before the final submission. This version should at least include the following:

- BigInteger implementation.
- Middle-square pseudorandom number generator implementation.
- At least one of the sorting algorithms.

Not completing the preliminary submission on time results in a 50% reduction of this assignment's final grade.